

MBA IN SOFTWARE ENGINEERING

Test Management (TDD) I-II

Prof. Helder Prado Santos

The trustworthiness, originality,
and legality of the educational content presented is responsibility of the professor.

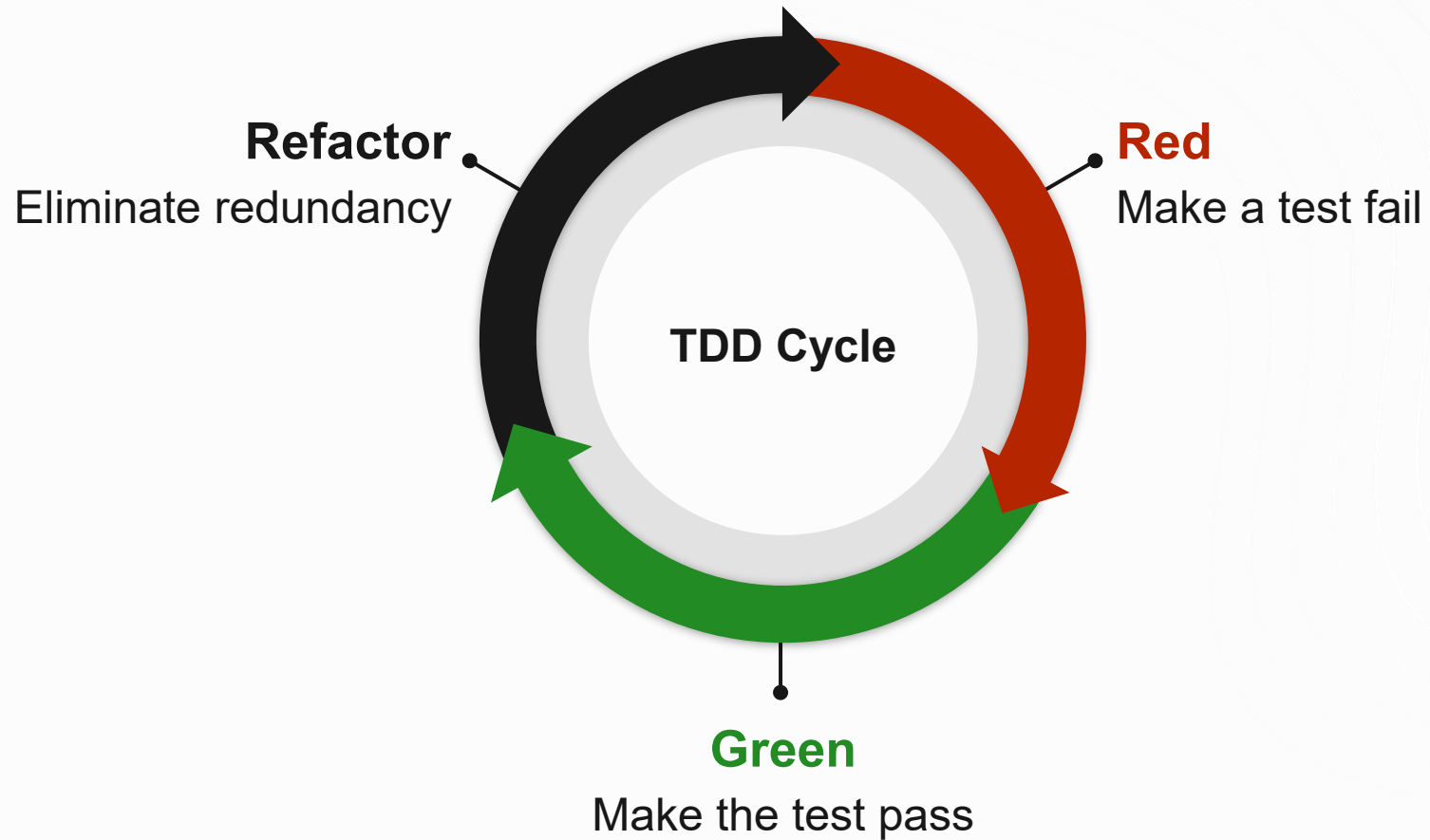
The total or partial **reproduction** of this material without authorization is prohibited.

Law No.
9610/98

Test-driven development (TDD)



TDD lifecycle



Objectives and Impacts

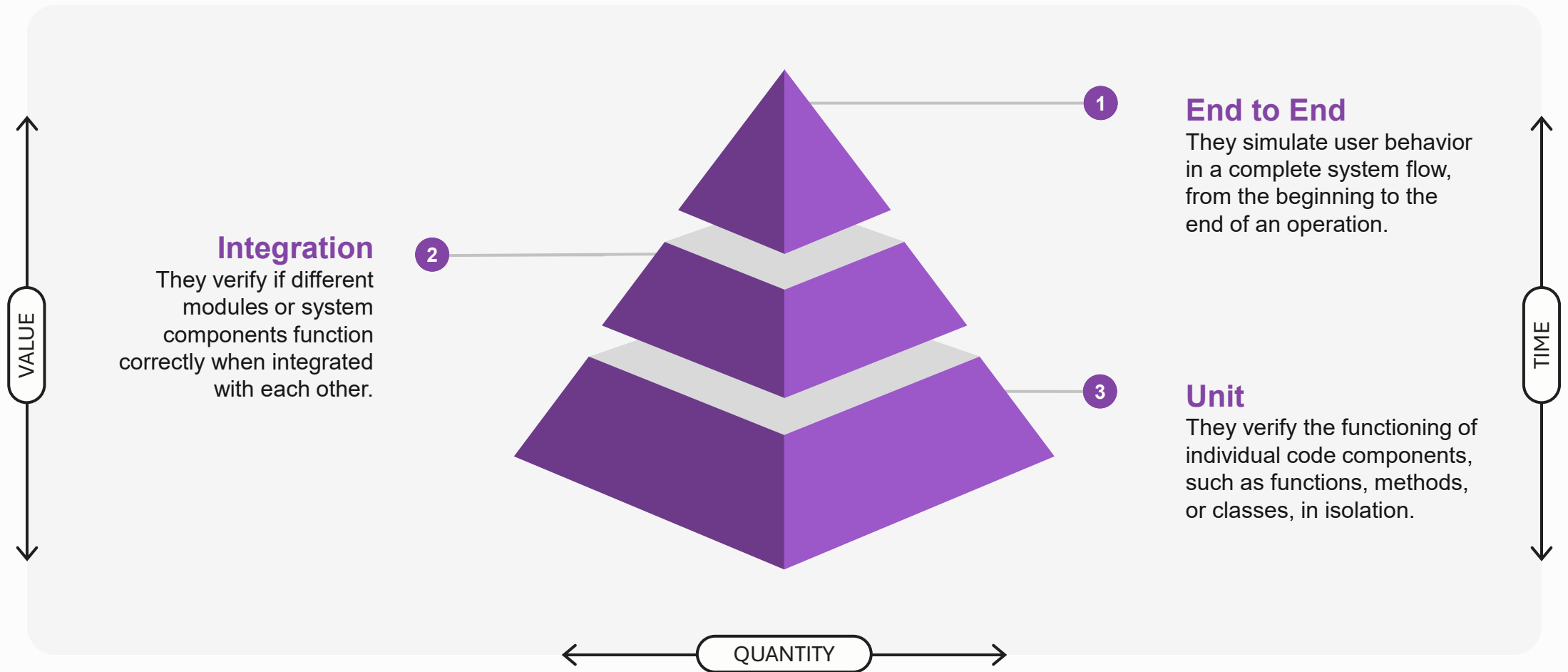
TDD Objectives:

- Reduce uncertainty and anxiety in software development.
- Increase code quality and reliability from the initial stages.
- Promote more agile and secure development.

Impact of Fear at Work:

- Excessive fear leads to hesitant programmers, with less proactivity and less openness to collaboration.
- It can affect motivation, causing frustration and blocking learning and creativity.

Test pyramid



Unit Tests

- **What they are:** Unit tests verify the functioning of individual code components, such as functions, methods, or classes, in isolation.
- **Objective:** Ensure that each unit of code functions correctly independently.
- **Focus:** Test a single part of the code, without dependence on other parts or external systems, such as databases.

Example: Test the methods of a class, such as the validation of its attributes.

Unit Tests

Advantages:

- Fast execution, as they test only small parts of the code.
- Easy identification of where the problem lies when a test fails, as each test focuses on a single component.

Common Tools:

- JUnit (Java)
- NUnit (.NET)
- Jest (JavaScript)
- PyTest (Python)

Integration Tests

- **What they are:** Integration tests verify if different modules or system components function correctly when integrated with each other.
- **Objective:** Ensure that the interaction between code modules (for example, interaction between different entities, or between an application and its database) occurs as expected.
- **Focus:** Test the system's functioning as a whole, generally including dependencies between components.

Example: Test the functionality where two entities interact.

Integration Tests

Advantages:

- Identification of integration problems that unit tests cannot cover, such as communication errors between modules.
- Useful for verifying compatibility between different parts of the system that need to operate together.

Common Tools:

- Spring Test (Java)
- DBUnit (for database testing)
- Mockito (for dependency simulation).

End-to-end tests (E2E)

- **What they are:** End-to-end (E2E) tests verify the functioning of an application in a complete flow, as a whole, to ensure that the integrated components provide the expected response to the client.
- **Objective:** Ensure that the system functions as a whole, validating the software's behavior from the end-user's perspective.
- **Focus:** Test the entire application, simulating how the system responds to real operations.
- **Example:** Test a login endpoint, sending a user authentication request and verifying if the system responds successfully.

End-to-End Tests (E2E)

Advantages:

- Confidence that the system is ready for the end-user, as it covers the complete flow.
- Ability to identify integration problems between systems and complex flows that depend on multiple services and components.

Common Tools:

- Postman
- Cypress
- Swagger

Test Life Cycle

- Test planning and specification
- Test development and implementation
- Execution and results logging
- Failure analysis and monitoring



Test Planning and Strategy

- Definition of acceptance criteria and test coverage;
- Test case planning and test scope;
- Risks and test prioritization criteria;
- Resource allocation and effort estimation.

Best Practices in TDD

- Keep tests independent and atomic
- Clear and descriptive test naming
- Avoid excessive dependencies and coupling in tests
- Refactor frequently to avoid "test code smells"

Test Coverage and Metrics

What is Test Coverage?

- Measures how much of the code is executed during test execution.
- Essential indicator to evaluate the effectiveness and breadth of tests.
- Helps identify uncovered areas that may contain defects.

Quality Metrics for Tests and Results Analysis

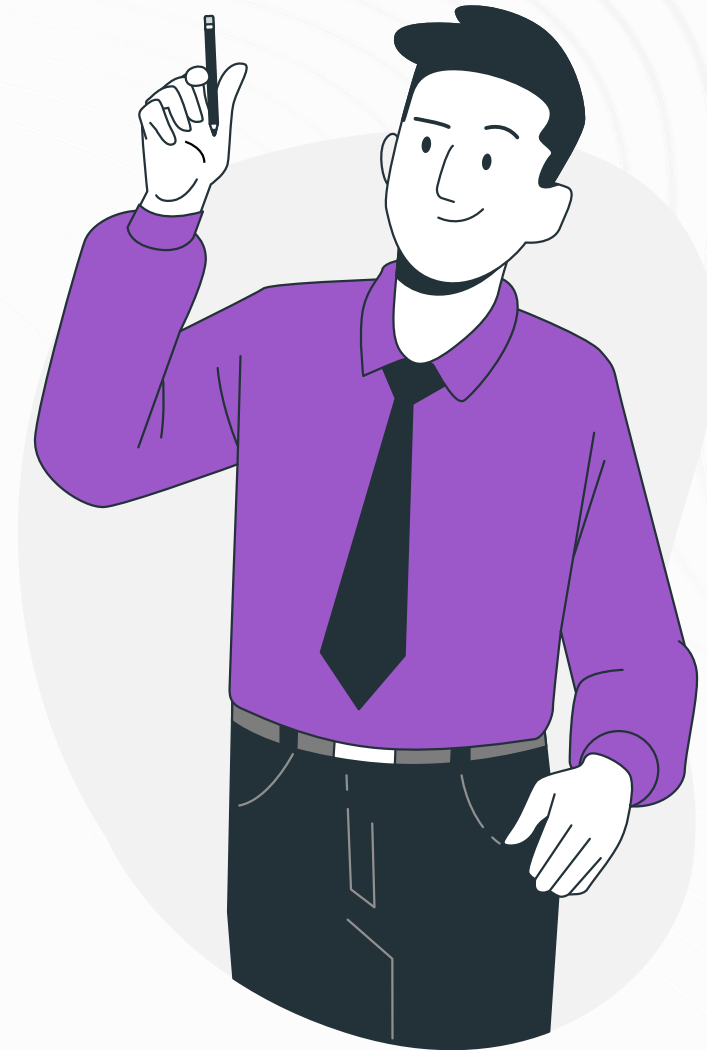
- **Coverage Rate:** Percentage of coverage considered adequate for the application.
- **Defect Metrics:** Number and type of defects found in covered and uncovered areas.
- **Effectiveness Indicators:** Relationship between coverage and defects, showing the accuracy of tests in identifying errors.
- **Trend Over Time:** Analysis of the evolution of test coverage and quality in each release.

Tools for Code Coverage Analysis

- **Jacoco** (Java), **Istanbul** (JavaScript), **Coverage.py** (Python)
- **SonarQube**: For continuous code quality analysis and integration with CI/CD tools
- **Codecov and Coveralls**: For visualization and monitoring of code coverage in open-source projects and CI/CD

Defect Management

- Defect identification and categorization process
- Defect life cycle: Detection, analysis, correction, and verification
- Defect management tools (e.g., Jira, Bugzilla)
- Relationship between defects and test coverage



Tests as System Documentation

Tests as a Source of Living Documentation

- Tests, especially automated ones, offer a clear view of how the system should behave in different scenarios.
- They serve as automatically updated documentation: whenever the code is changed, the tests reflect these changes.

Advantages of Tests as Documentation

- **Executable Specification:**

Tests show how each functionality should operate, replacing manual descriptions.

- **Continuous Update:**

Each test execution verifies and maintains the system's functional documentation.

- **Understanding System Behavior:**

Unit, integration, and end-to-end tests explain the logic and interactions of components in real situations.

TDD cycle example

```
1  import pytest
2  from uuid import uuid4
3
4  class TestUser:
5
6      # Teste para inicialização do usuário
7      def test_user_initialization(self):
8          user_id = uuid4()
9          user_name = "Test User"
10         user = User(id=user_id, name=user_name)
11
12         assert user.id == user_id
13         assert user.name == user_name
14
```



TDD cycle example

```
1  from uuid import UUID
2
3  class User:
4      |
5      |     id: UUID
6      |     name: str
7      |
8      |     def __init__(self, id: UUID, name: str):
9      |         |
10         |         self.id = id
10         |         self.name = name
```

TDD cycle example

```
1  import pytest
2  from uuid import uuid4
3  from domain.user.user_entity import User
4
5
6  class TestUser:
7
8      # Teste para inicialização do usuário
9      def test_user_initialization(self):
10         user_id = uuid4()
11         user_name = "Test User"
12         user = User(id=user_id, name=user_name)
13
14         assert user.id == user_id
15         assert user.name == user_name
16
```



TDD cycle example

```
1  import pytest
2  from uuid import uuid4
3  from domain.user.user_entity import User
4
5  class TestUser:
6
7      # Teste para inicialização do usuário
8      def test_user_initialization(self):
9          user_id = uuid4()
10         user_name = "Test User"
11         user = User(id=user_id, name=user_name)
12
13         assert user.id == user_id
14         assert user.name == user_name
15
16     # Teste para validação do ID do usuário
17     def test_user_id_validation(self):
18         with pytest.raises(Exception, match="id must be an UUID"):
19             User(id="invalid_id", name="Test User")
20
21     # Teste para validação do nome do usuário
22     def test_user_name_validation(self):
23         user_id = uuid4()
24         with pytest.raises(Exception, match="name is required"):
25             User(id=user_id, name="")
```



TDD cycle example

```
1  from uuid import UUID
2
3  class User:
4
5      id: UUID
6      name: str
7
8      def __init__(self, id: UUID, name: str):
9          self.id = id
10         self.name = name
11         self.validate()
12
13     def validate(self):
14         if not isinstance(self.id, UUID):
15             raise Exception("id must be an UUID")
16
17         if not isinstance(self.name, str) or len(self.name) == 0:
18             raise Exception("name is required")
```

TDD cycle example

```
1 import pytest
2 from uuid import uuid4
3 from domain.user.user_entity import User
4
5 class TestUser:
6
7     # Teste para inicialização do usuário
8     def test_user_initialization(self):
9         user_id = uuid4()
10        user_name = "Test User"
11        user = User(id=user_id, name=user_name)
12
13        assert user.id == user_id
14        assert user.name == user_name
15
16    # Teste para validação do ID do usuário
17    def test_user_id_validation(self):
18        with pytest.raises(Exception, match="id must be an UUID"):
19            User(id="invalid_id", name="Test User")
20
21    # Teste para validação do nome do usuário
22    def test_user_name_validation(self):
23        user_id = uuid4()
24        with pytest.raises(Exception, match="name is required"):
25            User(id=user_id, name="")
```



Thank you!

Prof. Helder Prado Santos | <https://www.linkedin.com/in/helderprado>

MBA **USP**
ESALQ